



**UNIVERSITY  
OF LONDON**

University of London  
Computer Science

Nicolas Conde Salazar

## **Mid-term Course Work**

**Advanced Web Development**

CM3035

January / 7 / 2026

# 1 Introduction

This report documents a RESTful API built with Django and Django REST Framework (DRF) for the Los Angeles Inside Airbnb dataset. The application provides comprehensive CRUD operations for property listings, analytical endpoints for market insights, and JWT-based authentication to protect write operations. The architecture follows REST conventions with proper resource nesting, input validation, and OpenAPI documentation.

## 2 Dataset Overview

The application uses the Inside Airbnb dataset for Los Angeles, containing real-world short-term rental data. The dataset comprises two primary CSV files:

- **listings.csv**: Contains 5,000 Airbnb listings with host profiles, geographic coordinates, property details, pricing, and aggregated review metrics
- **reviews.csv**: Contains guest reviews linked to listings, including reviewer information and comment text

This dataset is particularly interesting for API development because it represents genuine market data with natural relationships between entities. The data enables meaningful queries such as price analysis across neighbourhoods, identification of top-performing hosts, seasonal review trends, and value-based property rankings. These real-world use cases demonstrate practical API design patterns beyond simple CRUD operations.

## 3 Data Model Design (R2)

The application implements three Django models that normalize the denormalized CSV data into a proper relational structure, establishing clear entity relationships and enforcing data integrity through constraints.

### 3.1 Entity Relationships

|          |                  |                 |
|----------|------------------|-----------------|
| Host (1) | < Listing (many) | < Review (many) |
|----------|------------------|-----------------|

A host can own multiple listings, and each listing can receive multiple reviews. This hierarchy reflects the real-world Airbnb domain model.

### 3.2 Host Model

```
class Host(models.Model):
    host_id = models.BigIntegerField(unique=True, db_index=True)
    name = models.CharField(max_length=255, blank=True, null=True)
    is_superhost = models.BooleanField(default=False)
    response_rate = models.FloatField(
        validators=[MinValueValidator(0), MaxValueValidator(100)]
    )
    total_listings_count = models.PositiveIntegerField(default=0)
```

The `host_id` field uses `BigIntegerField` to accommodate Airbnb's large numeric identifiers. Database indexing on this field enables efficient lookups during both data import and API queries. The `response_rate` validator ensures values remain within the valid 0-100 percentage range.

### 3.3 Listing Model

```
class Listing(models.Model):
    ROOM_TYPE_CHOICES = [
        ('Entire home/apt', 'Entire home/apt'),
        ('Private room', 'Private room'),
```

```

        ('Shared room', 'Shared room'),
        ('Hotel room', 'Hotel room'),
    ]

    listing_id = models.BigIntegerField(unique=True, db_index=True)
    host = models.ForeignKey(Host, on_delete=models.CASCADE,
                             related_name='listings')
    neighbourhood = models.CharField(max_length=255, db_index=True)
    room_type = models.CharField(max_length=50, choices=ROOM_TYPE_CHOICES,
                                 db_index=True)
    price = models.DecimalField(max_digits=10, decimal_places=2,
                                validators=[MinValueValidator(0)])
    number_of_reviews = models.PositiveIntegerField(default=0)

```

The `ROOM_TYPE_CHOICES` constraint ensures data integrity by restricting values to valid Airbnb room types. The `related_name='listings'` enables efficient reverse lookups such as `host.listings.all()`. Composite database indexes on frequently filtered fields (`room_type`, `neighbourhood`) optimize common query patterns.

### 3.4 Review Model

```

class Review(models.Model):
    listing = models.ForeignKey(Listing, on_delete=models.CASCADE,
                                related_name='reviews')
    review_id = models.BigIntegerField(unique=True, db_index=True)
    date = models.DateField(db_index=True)
    reviewer_name = models.CharField(max_length=255)
    comments = models.TextField()

```

The date field is indexed to support efficient time-based queries in the reviews-by-month statistics endpoint.

## 4 REST API Endpoints Implementation (R3)

The following sections detail six key endpoints that demonstrate the application's capabilities.

### 4.1 1. GET /api/listings/ - Search Listings

This endpoint returns a paginated list of listings with comprehensive filtering:

```

class ListingViewSet(viewsets.ModelViewSet):
    queryset = Listing.objects.select_related('host')
    permission_classes = [IsAuthenticatedOrReadOnly]
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_fields = ['room_type', 'neighbourhood', 'neighbourhood_group']
    ordering_fields = ['price', 'number_of_reviews', 'reviews_per_month']

```

The `select_related('host')` call optimizes database queries by fetching host data in a single SQL JOIN rather than issuing separate queries for each listing. DRF's filter backends enable query parameters like `?room_type=Private room&price_min=50&ordering=-price`. The `IsAuthenticatedOrReadOnly` permission allows public read access while requiring JWT authentication for modifications.

### 4.2 2. POST /api/listings/ - Create Listing

Creating a listing requires authentication and validates input through serializers:

```

class ListingCreateUpdateSerializer(serializers.ModelSerializer):
    host_id = serializers.IntegerField(write_only=True)

    def validate_price(self, value):

```

```

        if value <= 0:
            raise serializers.ValidationError('Price must be greater than 0.')
        return value

    def validate_host_id(self, value):
        if not Host.objects.filter(host_id=value).exists():
            raise serializers.ValidationError(f'Host with host_id {value} does
            not exist.')
        return value

```

The serializer validates that prices are positive and that the referenced host exists before creating the listing. Review-related fields (`number_of_reviews`, `reviews_per_month`, `last_review`) are defined as read-only since they are computed from actual review data rather than user input. The `listing_id` is auto-generated if not provided, finding the maximum existing ID and incrementing it.

### 4.3 3. POST /api/listings/{id}/reviews/ - Create Review

This endpoint creates a review as a nested resource under listings and automatically updates the listing's statistics:

```

def create(self, validated_data):
    listing = self.context.get('listing')

    # Auto-generate review_id
    max_id = Review.objects.order_by('-review_id').values_list('review_id',
        flat=True).first() or 0
    validated_data['review_id'] = max_id + 1
    validated_data['date'] = timezone.now().date()

    review = Review.objects.create(listing=listing, **validated_data)

    # Atomic update of listing statistics
    Listing.objects.filter(pk=listing.pk).update(
        number_of_reviews=F('number_of_reviews') + 1,
        last_review=review.date
    )

    return review

```

This design ensures data consistency, the listing's review count always reflects the actual number of reviews. The use of `F('number_of_reviews') + 1` performs an atomic database-level increment, preventing race conditions in concurrent requests.

### 4.4 4. GET /api/hosts/top/ - Top Hosts by Listing Count

This analytical endpoint aggregates data to rank hosts by their number of listings:

```

@action(detail=False, methods=['get'])
def top(self, request):
    queryset = Listing.objects.all()

    neighbourhood_group = request.query_params.get('neighbourhood_group')
    if neighbourhood_group:
        queryset = queryset.filter(neighbourhood_group=neighbourhood_group)

    limit = int(request.query_params.get('limit', 10))

    top_hosts = (
        queryset
        .values('host__host_id', 'host__name', 'host__is_superhost')
        .annotate(listing_count=Count('id'), avg_price=Avg('price'))
        .order_by('-listing_count')[:limit]
    )

```

```
)

return Response(TopHostSerializer(top_hosts, many=True).data)
```

The `annotate()` function performs COUNT and AVG aggregations at the database level, which is significantly more efficient than processing in Python. The optional `neighbourhood_group` filter enables analysis of top hosts within specific geographic areas.

## 4.5 5. GET /api/stats/price-by-neighbourhood/ - Price Statistics

This endpoint provides aggregate price statistics grouped by neighbourhood:

```
class PriceByNeighbourhoodView(APIView):
    def get(self, request):
        queryset = Listing.objects.all()

        room_type = request.query_params.get('room_type')
        if room_type:
            queryset = queryset.filter(room_type=room_type)

        stats = (
            queryset
            .values('neighbourhood')
            .annotate(
                listing_count=Count('id'),
                avg_price=Avg('price'),
                min_price=Min('price'),
                max_price=Max('price'),
            )
            .order_by('-listing_count')
        )

        return Response(PriceByNeighbourhoodSerializer(stats, many=True).data)
```

Using Django’s ORM aggregation functions (Avg, Min, Max, Count) pushes computation to the database engine, making this endpoint performant even with thousands of listings. The optional `room_type` filter enables comparisons such as “average price for entire homes versus private rooms in Venice.”

## 4.6 6. GET /api/stats/reviews-by-month/ - Review Activity Over Time

This endpoint tracks review activity trends using date truncation:

```
class ReviewsByMonthView(APIView):
    def get(self, request):
        queryset = Review.objects.all()

        from_date = request.query_params.get('from')
        to_date = request.query_params.get('to')
        if from_date:
            queryset = queryset.filter(date__gte=from_date)
        if to_date:
            queryset = queryset.filter(date__lte=to_date)

        stats = (
            queryset
            .annotate(month=TruncMonth('date'))
            .values('month')
            .annotate(review_count=Count('id'))
            .order_by('month')
        )

        return Response(ReviewsByMonthSerializer(stats, many=True).data)
```

The `TruncMonth` function groups reviews by month at the database level. This endpoint reveals seasonal patterns in Airbnb activity, valuable for hosts planning availability or analysts studying market dynamics.

---

## 5 Django REST Framework Implementation (R1)

### 5.1 Models and Migrations (R1a)

Models use Django's ORM with appropriate field types, validators, and relationships. Migrations are auto-generated and applied:

```
python manage.py makemigrations
python manage.py migrate
```

### 5.2 Validators and Serialization (R1b)

Different serializers serve different purposes: `ListingListSerializer` provides compact list views with essential fields, while `ListingDetailSerializer` includes full details with nested host data. Custom validation methods ensure business rules are enforced at the API layer.

### 5.3 Django REST Framework Usage (R1c)

The application uses DRF's `ModelViewSet` for standard CRUD operations and `APIView` for custom analytical endpoints. The `@action` decorator creates nested resources like `/listings/{id}/reviews/`.

### 5.4 URL Routing (R1d)

```
router = DefaultRouter()
router.register(r'hosts', HostViewSet, basename='host')
router.register(r'listings', ListingViewSet, basename='listing')

urlpatterns = [
    path('', include(router.urls)),
    path('stats/price-by-neighbourhood/', PriceByNeighbourhoodView.as_view()),
    path('stats/reviews-by-month/', ReviewsByMonthView.as_view()),
]
```

The router automatically generates RESTful URL patterns for CRUD operations.

### 5.5 Unit Testing (R1e)

The test suite contains 28 tests covering models, API endpoints, filtering, validation, and authentication:

```
def test_filter_by_room_type(self):
    response = self.client.get('/api/listings/', {'room_type': 'Private room'})
    self.assertEqual(response.status_code, status.HTTP_200_OK)
    for listing in response.data['results']:
        self.assertEqual(listing['room_type'], 'Private room')
```

Tests verify that filters return only matching results across all items, that authentication is properly enforced, and that validation rejects invalid data.

---

## 6 Bulk Data Loading (R4)

The `import.airbnb.data` management command efficiently loads CSV data:

```

class Command(BaseCommand):
    def handle(self, *args, **options):
        self._load_hosts(listings_file)
        self._load_listings(listings_file)
        self._load_reviews(reviews_file)

    def _parse_price(self, price_str):
        """Parse price string like '$185.00' to Decimal."""
        cleaned = re.sub(r'[^\\d.]', '', price_str)
        return Decimal(cleaned) if cleaned else Decimal('0')

```

The command uses `update_or_create()` for upsert logic, updating existing records or creating new ones. Helper methods parse price strings, booleans (t/f), and handle missing values gracefully. The `--clear` flag allows wiping existing data before import for clean re-runs.

---

## 7 Authentication and Security

The API implements JWT (JSON Web Token) authentication using `django-rest-framework-simplejwt`. This approach provides stateless authentication suitable for REST APIs:

```

# settings.py
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
}

```

Three authentication endpoints are exposed:

- **POST** `/api/auth/register/`: Creates a new user account with username and password
- **POST** `/api/auth/login/`: Returns access and refresh JWT tokens
- **POST** `/api/auth/refresh/`: Exchanges a refresh token for a new access token

The `IsAuthenticatedOrReadOnly` permission class is applied to resource endpoints, allowing anonymous users to browse listings and statistics while requiring valid JWT tokens for create, update, and delete operations. This design balances accessibility with data protection.

---

## 8 API Documentation

The application uses `drf-spectacular` to generate OpenAPI 3.0 documentation automatically from the codebase. The `@extend_schema_decorator_enriches_auto-generated_documentation_with_descriptions, parameter_details, and`

```

@extend_schema(
    tags=['listings'],
    summary='Create a new listing',
    description='Create a new Airbnb listing. Requires authentication.',
    examples=[
        OpenApiExample(
            'Complete listing',
            value={'host_id': 107434423, 'room_type': 'Entire home/apt',
                  'price': 185.00},
            request_only=True,
        ),
    ],
)

```

Interactive documentation is available at `/api/docs/` (Swagger UI) and `/api/redoc/` (ReDoc), enabling developers to explore and test endpoints directly in the browser.

---

## 9 Setup and Running Instructions

### 9.1 Package Versions

| Package                       | Version |
|-------------------------------|---------|
| Django                        | 6.0     |
| djangorestframework           | 3.16.1  |
| djangorestframework-simplejwt | 5.5.1   |
| django-filter                 | 25.2    |
| drf-spectacular               | 0.29.0  |

### 9.2 Development Environment

- Operating System: macOS (Darwin)
- Python Version: 3.14.0

### 9.3 Installation Steps

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py import_airbnb_data
python manage.py createsuperuser
python manage.py runserver
```

### 9.4 Django Admin Access

- URL: <http://localhost:8000/admin/>
- Credentials: Use the username and password created with `createsuperuser`

### 9.5 Running Unit Tests

```
python manage.py test core
```

### 9.6 Data Loading Script Location

`core/management/commands/import_airbnb_data.py`

### 9.7 Interactive API Documentation URLs

- Swagger UI: <http://localhost:8000/api/docs/>
- ReDoc: <http://localhost:8000/api/redoc/>

## 10 Conclusion

This application demonstrates a complete REST API implementation using Django and Django REST Framework. The architecture separates concerns appropriately: models define the data structure with proper constraints and relationships, serializers handle validation and data transformation, and views implement business logic with efficient database queries. The combination of standard CRUD endpoints and analytical statistics endpoints showcases both typical API patterns and more advanced aggregation techniques. JWT authentication protects write operations while maintaining public read access, and comprehensive unit tests ensure reliability. The bulk data loading command enables reproducible database population from the original CSV dataset.